

Applied A.I. Solutions

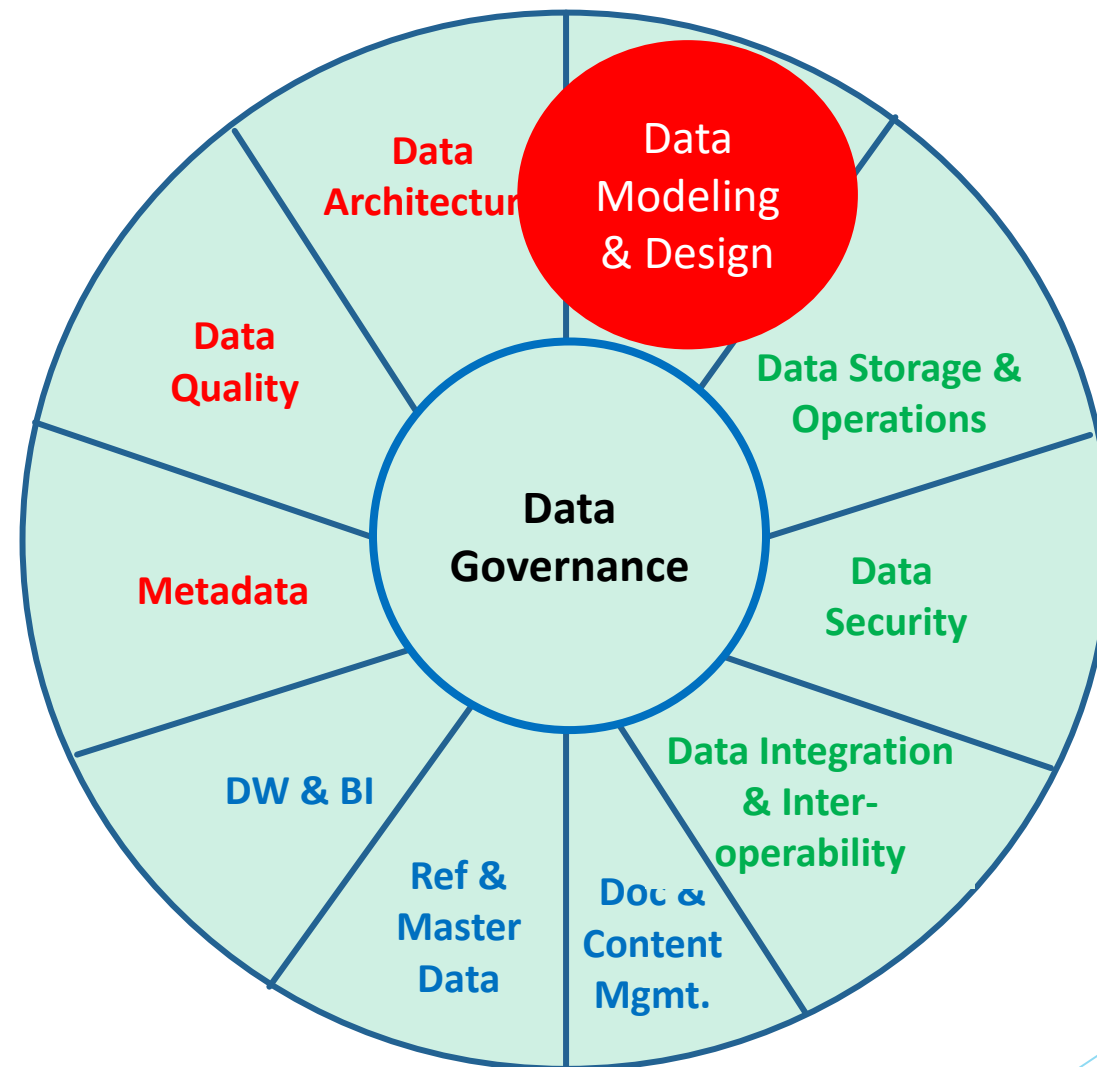
Foundations of Data Management

Professor
Daniel Vitaver, B.Sc., EMBA

daniel.vitaver@georgebrown.ca

DATA MODELING AND DESIGN

The DAMA Wheel



INTRODUCTION

- **Data Models are a critical component** of DM and enable organizations to understand its data assets
- Common schema are **relational, dimensional**, object-Oriented, NoSQL
- Models exist at a **conceptual, logical, and physical level**.
- Each model contains a set of components: entities, relationships, facts, keys, and attributes

DM&D Framework

Definition

Data Modeling is the process of discovering, analyzing, and scoping data requirements, and then **representing** and **communicating** these data requirements in a **precise** form called the **Data Model**.

Principles

- **Formalization** (data model **documents**)
- Scope Definition (**boundaries** for data context)
- Knowledge retention / documentation

Goals

To **confirm and document** an understanding of different perspectives, which leads to applications that align with business requirements



Business Driver

Inputs

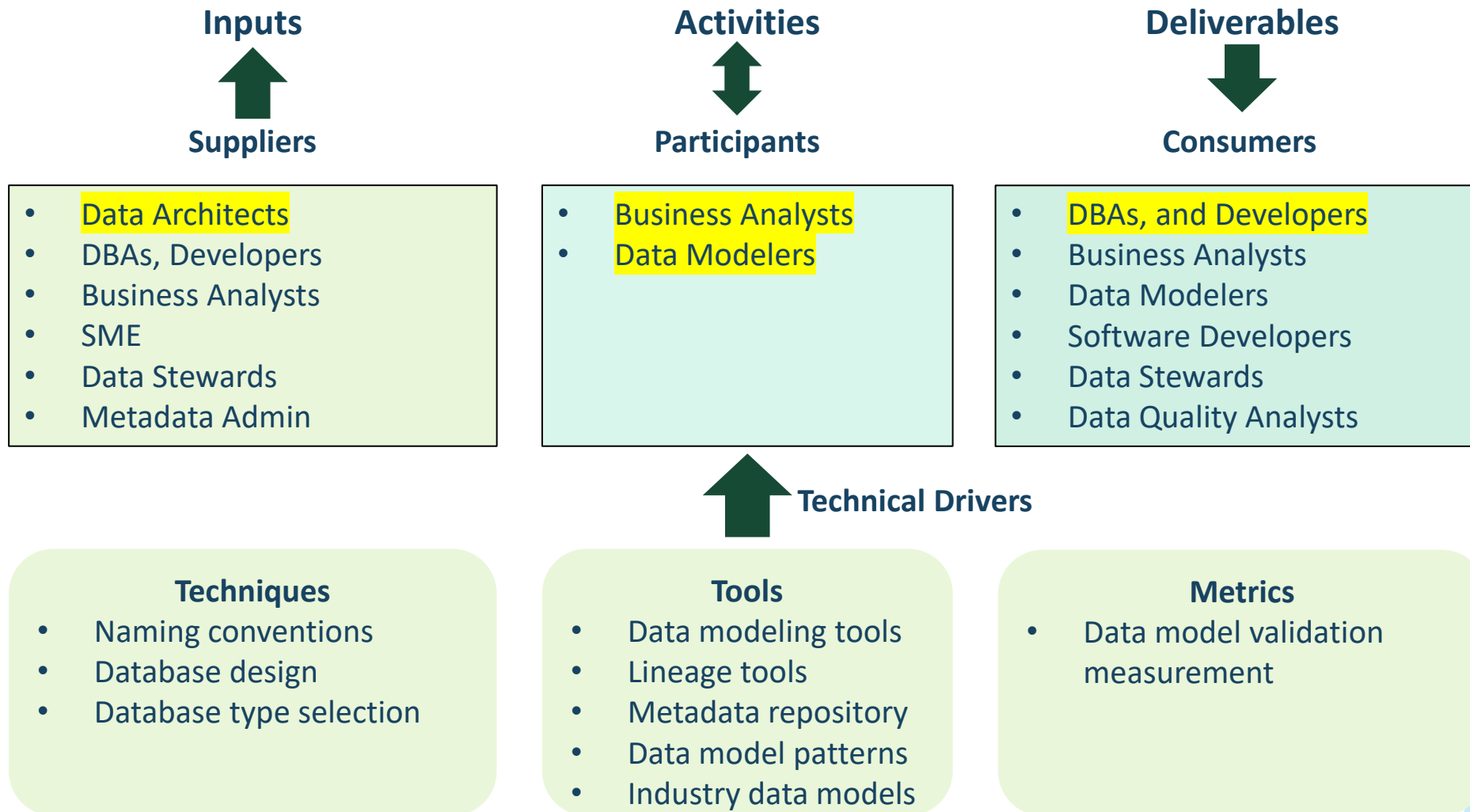
- **Data architecture**
- Existing data models
- Data standards
- Data sets
- Initial data requirements
- Original data requirements

Activities

1. Plan for Data Modeling
2. **Build the Data Models**
(conceptual, logical, physical)
3. Review the Data Models
4. Manage the Data Model

Deliverables

1. Data Models
 - – conceptual
 - – logical
 - – physical



Data Models

- Provide a **common vocabulary** around data
- Capture and document explicit **knowledge** about an organization's data and systems
- Serve as primary **communication tool** during projects
- Data Models contains **4 main building blocks**:
 - Entities
 - Relationships
 - Attributes
 - Domains

Entities

- Entity is a thing that exists “**per se**”, separate from other things
- It is a thing about which the organization collects information
- They are referred to as **nouns**
- The entity represents the answer to:
 - Who
 - What
 - When, Where, Why
 - How

Relationship

- It captures the high-level interactions or association between conceptual entities (Chen, 1976)
- The detailed interactions between logical entities
- The constraints between physical entities

Relationship Cardinality

- It captures how many entity-instances participate in a relationship with how many of the other entity
- The most common form of notation is Information Engineering (IE) syntax (crow's foot)

Crow's foot notation

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many

Attributes

- An attribute is a property that **identifies, describes, or measures** an entity.
- The physical correspondent of an **attribute** in an entity is a **column**, field or tag, or node in a table, view, document, graph or file

Keys

- Are a set of one or more **attributes that uniquely** defined an instance of an entity

Primary Key

- It **uniquely identifies a record** in the relational database table
- A primary key is used to ensure data in the specific column is unique
- Only one primary key is allowed in a table
- It does not allow NULL
- Values cannot be deleted from the parent table.
- Examples of primary keys include **Unique last name, Social security number, online username**

Secondary Key

- A secondary key provides a **secondary reference point** for objects whose primary keys do not adequately distinguish them for reference purposes
- It is used for identification of rows but not usually unique
- We can have multiple secondary key per table
- Allows NULL values.
- Examples of secondary keys include street address number, phone number, middle name

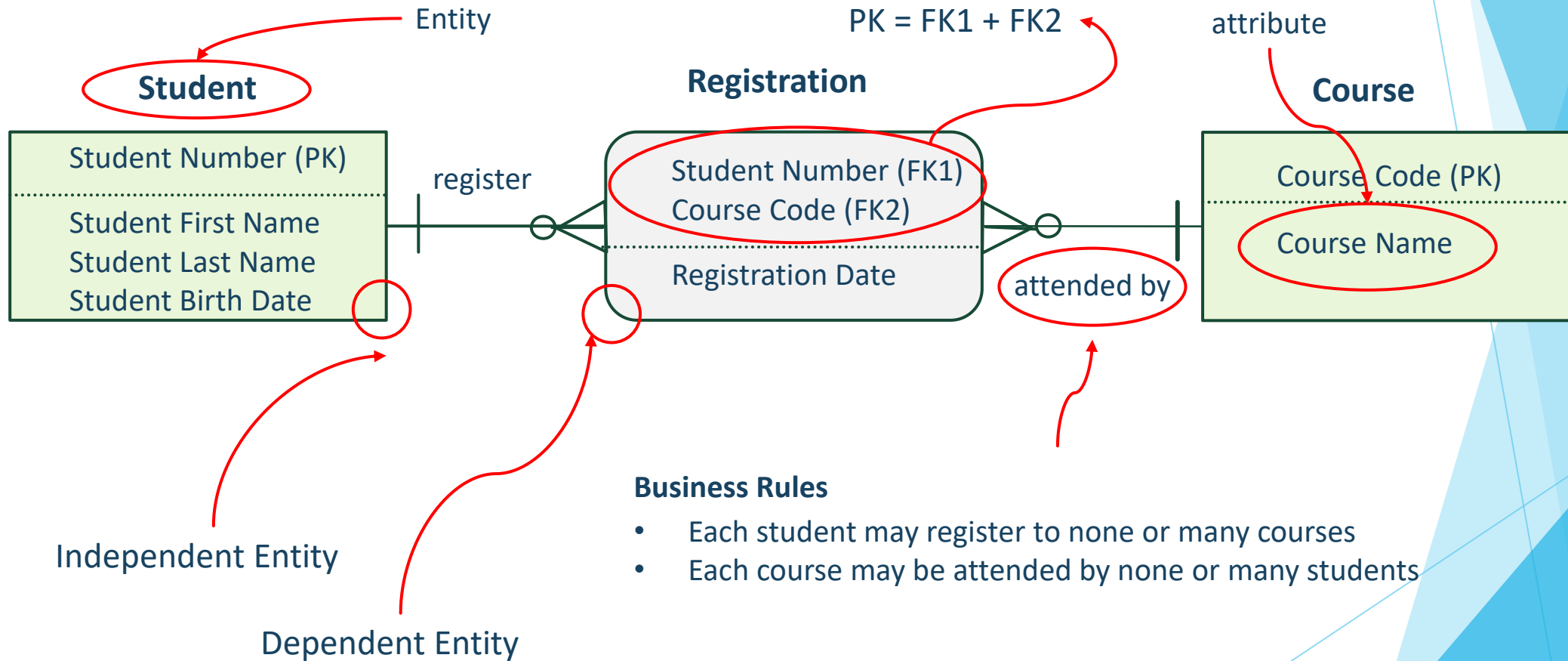
Foreign Key

- A foreign key is used in physical and sometimes logical relational data modeling schemes to represent relationships among entities
- **A foreign key is created implicitly when a relationship is defined between entities**, depending on the database technology or the data modeling tools, and whether the two entities have mutual dependencies

Domain

- A complete set of possible values that an attribute can be assigned.
- Provides a means of **standardizing** the characteristics of the attributes
- All values inside a domain are **valid** values, outside are invalid
- Attributes should not contain values **outside** its domain
- Domain constraints are specific rules that restrict a domain
- Domains can be defined by data type, format, list, range, rule-based

Logical Level

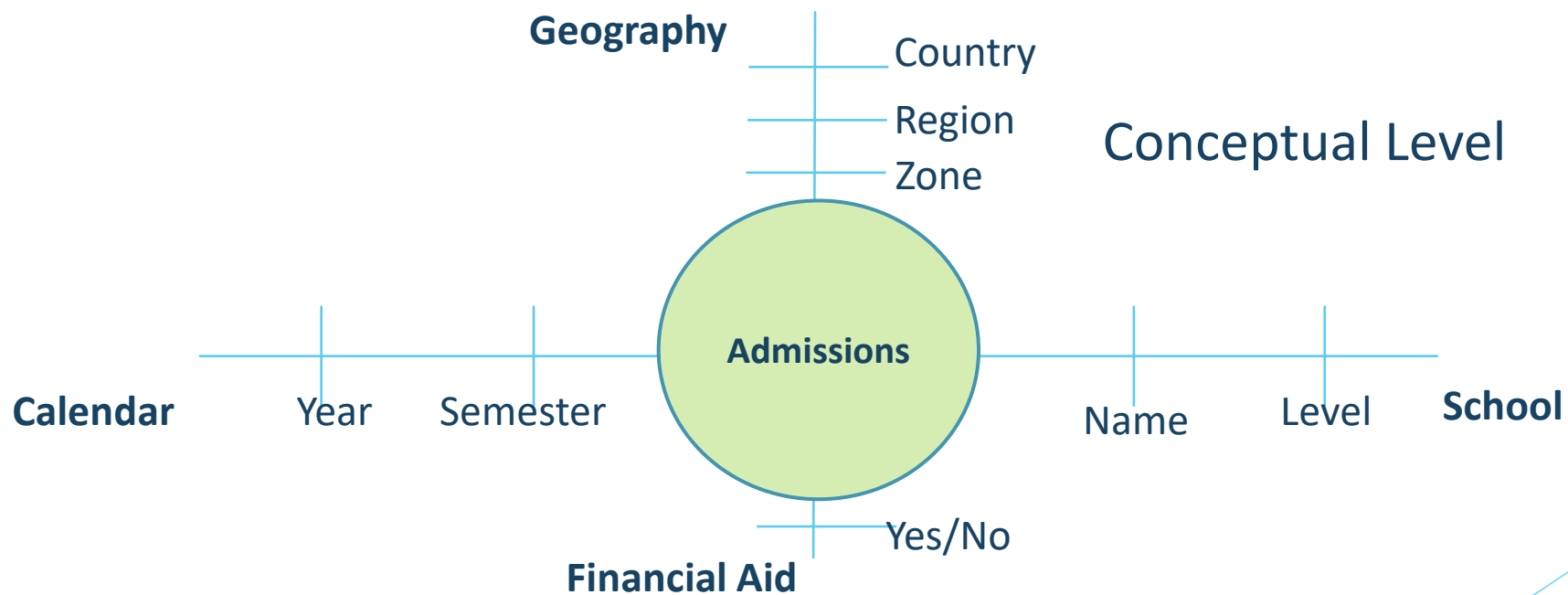


Relational Scheme

- Relational Theory provides a **systemic way to organize data** so that they reflected their meaning (Codd, 1970)
- The design objective is to have an exact expression of business data and to have **one fact in one place** (with no redundancies)
- It is ideal for the design of **operational systems** (transactional database)

Dimensional Scheme

- Data is structured to optimize the query and analysis
- Dimensional models focus on a particular business process



Normalization

Group-study and Self-study

Normalization

- Rules to organize business complexity into stable data structures
- Keeps attributes in only one place to eliminate redundancy, inconsistencies
- Rules sort attributes according to the PK and FKs

Pros and Cons

- Increases data **consistency** as it avoids the duplicity
- Helps in grouping related data under the **same schema**
- Improves searching **faster** as indexes can be created faster (OLTP)
- **Delays** the retrieving of data as more table joins are needed (performance)
- Normalization is not a good option in **OLAP** transactions

- **1NF:** each entity has a valid PK, every attribute depends on the PK
- **2NF:** each entity has the minimal PK, every attribute depends on the complete PK
- **3NF:** each entity has no hidden PKs; each attribute depends on no attributes outside the whole key

- **Boyce / Codd normal form (BCNF):** resolves overlapping composite candidate keys
- **4NF:** resolves many-to-many-to-many relationships (and beyond) in pairs until entities cannot be broken down into smaller pieces
- **5NF:** resolves inter-entity dependencies into basic pairs, and all join dependencies use parts of the PK

Normalization – cont'd

1NF

EmployeeNumber	LastName	FirstName	AreaName	AreaCity	AreaCountry
1111	Andrews	Jack	Accounting	Toronto	Canada
1115	Smith	Mike	Technology	Toronto	Canada
1220	Jones	Harry	HR	New York	USA
1250	Harvey	John	Admin	London	UK
1250	Harvey	John	HR	London	UK

2NF

EmployeeNumber	LastName	FirstName
1111	Andrews	Jack
1115	Smith	Mike
1220	Jones	Harry
1250	Harvey	John
1250	Harvey	John

Table A

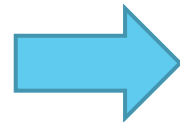
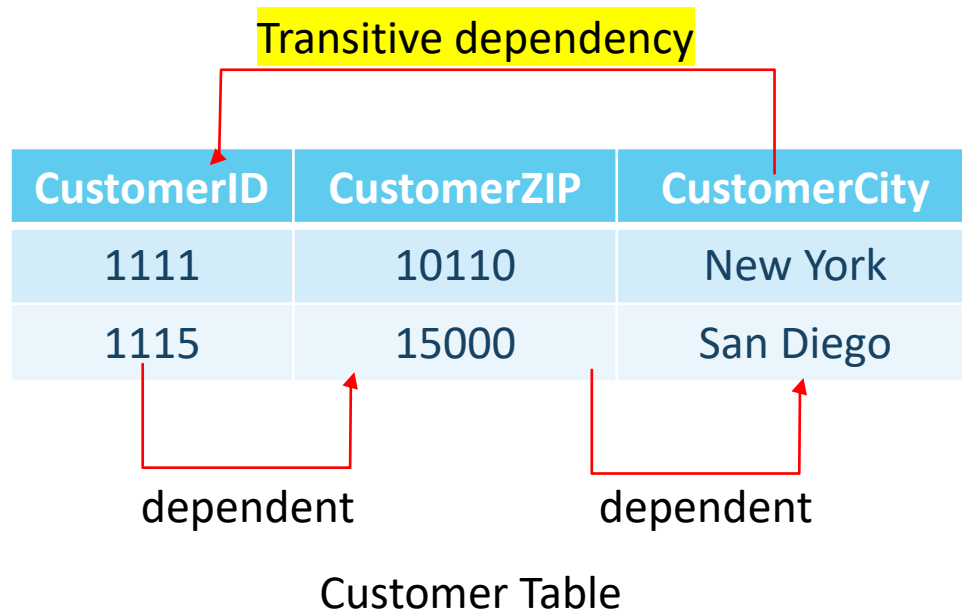
EmpAreaID	EmployeeNumber	AreaNumber
1	1111	10
2	1115	20
3	1220	30
4	1250	40
5	1250	30

Table C

AreaNumber	AreaName	AreaCity	AreaCountry
10	Accounting	Toronto	Canada
20	Technology	Toronto	Canada
30	HR	New York	USA
40	Admin	London	UK

Table B

3NF



Customer Table

CustomerID	CustomerZIP
1111	10110
1115	15000

CustZIP Table

CustomerZIP	CustomerCity
10110	New York
15000	San Diego

